

1. SRP – Single Responsibility Principle (Princip jedne odgovornosti)

Svaka klasa u modelu ima jasno definisanu i ograničenu odgovornost.

- **Osoba (i podtipovi):** Klasa `Osoba` i njeni nasljednici (`Student`, `Tutor`, `SystemManager`) bave se isključivo podacima o korisnicima i njihovim specifičnim akcijama unutar sistema.
- **Termin:** Zadužena je isključivo za logiku zakazivanja, statusa i vremena održavanja instrukcija.
- **Transakcija:** Upravlja isključivo procesom plaćanja i statusom novčanih tokova, bez miješanja u pedagoški dio procesa.
- **Recenzija:** Odgovorna je samo za čuvanje ocjena i povratnih informacija nakon održanog termina.

2. OCP – Open/Closed Principle (Princip otvorenosti/zatvorenosti)

Sistem je dizajniran tako da se može proširivati bez modifikacije postojećeg koda.

- **Nasljeđivanje:** Upotreba apstraktne klase `Osoba` omogućava dodavanje novih tipova korisnika u budućnosti bez mijenjanja logike u osnovnoj klasi ili povezanim klasama poput `Obavjestenje`.
- **Enumeracije:** Korištenje enumeracija kao što su `TipInstrukcija`, `StatusTermina` i `NacinPlacanja` omogućava definisanje fiksni stanja, dok se logika ponašanja može proširivati kroz nove metode unutar pripadajućih klasa.

3. LSP – Liskov Substitution Principle (Princip Liskovljeve zamjene)

Sve izvedene klase mogu zamijeniti osnovnu klasu bez narušavanja funkcionalnosti sistema.

- **Specijalizacija:** `Student`, `Tutor` i `SystemManager` nasljeđuju `Osobu` i implementiraju njene osnovne atribute (`ime`, `prezime`, `email`), ali ne mijenjaju osnovno značenje tih atributa.
- **Konzistentnost:** Metoda `posalji(primalac: Osoba)` u klasi `Obavjestenje` može primiti bilo koji objekt tipa `Student` ili `Tutor` jer oba ispunjavaju ugovor definisan u klasi `Osoba`.

4. ISP – Interface Segregation Principle (Princip segregacije interfejsa)

Iako su u dijagramu prikazane klase, vidljiva je jasna podjela metoda prema ulogama korisnika, što sprječava da klase zavise od metoda koje ne koriste.

- **Specifične funkcionalnosti:** `Student` ima metode za rezervaciju i plaćanje (`rezervisiTermin`, `izvrsiPlacanje`), dok `Tutor` ima metode za upravljanje terminima i prihode (`dodajTermin`, `prikaziStatistikuPrihoda`).
- **Administracija:** `SystemManager` posjeduje metode za upravljanje korisnicima i recenzijama koje obični korisnici (`Student/Tutor`) ne vide i ne mogu koristiti.

5. DIP – Dependency Inversion Principle (Princip inverzije zavisnosti)

Zavisnosti su usmjerene ka apstrakcijama gdje god je to moguće.

- **Apstrakcija Osobe:** Klasa `Obavjestenje` zavisi od apstrakcije `Osoba`, a ne direktno od konkretnih klasa `Student` ili `Tutor`, čime se smanjuje spregnutost (coupling) sistema.
- **Asocijacije:** Veze između klasa su definisane tako da klase međusobno komuniciraju putem identifikatora ili referenci na objekte, omogućavajući lakšu zamjenu implementacija u budućem razvoju (npr. promjena načina generisanja QR koda u klasi `Termin` ne utiče na klasu `Student`).